

软件概要设计说明书

1 引言

1.1 编写目的

本文档旨在对生活助手平台的系统设计进行概要说明，明确系统的整体架构、模块划分、接口设计、数据结构设计和运行设计，为后续的详细设计、编码实现和测试工作提供依据。预期读者包括项目开发成员、指导教师及课程验收人员。

1.2 背景

- 待开发软件系统的名称：生活助手平台
- 任务提出者：课程项目组
- 开发者：课程项目组成员
- 用户：普通消费者、平台商家、配送人员及平台管理员
- 运行该软件的计算站（中心）：校园实验服务器或普通云服务器，客户端为个人计算机浏览器

1.3 定义

- SDP：软件开发计划（Software Development Plan）
- SRS：软件需求规格说明书（Software Requirements Specification）
- CSCI：计算机软件配置项（Computer Software Configuration Item），本项目指生活助手平台软件系统
- JWT：JSON Web Token，用于身份认证的令牌
- RESTful：一种面向资源的Web服务架构风格
- 前后端分离：前端负责界面和交互，后端负责业务逻辑和数据处理，通过API通信的系统设计方式
- ORM：对象关系映射（Object-Relational Mapping）

1.4 参考设计

- 《生活助手平台软件开发计划书（SDP）》V1.0
- 《生活助手平台软件需求规格说明书（SRS）》V1.0
- 《数据库设计说明书》（待编制）
- 美团、饿了么、大众点评等参考系统业务分析资料

2 系统需求概述

2.1 需求规定

本系统的主要输入输出项目及功能性能要求如下：

- 输入**：用户注册信息、搜索关键词、订单信息、支付确认、评价内容、商家入驻信息、后台审核操作等。
- 输出**：商家列表、商品详情、订单状态、支付结果、评价展示、统计图表等。
- 功能**：用户注册登录、商家浏览与搜索、商品展示与购物车、外卖下单与团购下单、模拟支付、订单管理、用户评价、商家后台管理、配送员功能、管理员审核与统计。
- 性能**：简单查询响应时间小于1秒；支付接口响应时间小于500ms；支持至少10个并发用户；系统稳定运行时间不低于2小时（演示期间）。

2.2 业务目标

构建一个集外卖订餐、到店团购、休闲娱乐、生活服务于一体的本地生活服务平台，实现用户与本地商家的高效连接，形成“吃、喝、玩、乐、购”一站式服务闭环，同时为商家提供线上经营渠道，为管理员提供监管工具。

2.3 运行环境

2.3.1 设备

- **客户端设备：**普通台式机或笔记本电脑，支持Chrome 90+或Edge 90+浏览器。
- **服务器端设备：**CPU 1核以上，内存2GB以上，硬盘20GB以上的云服务器或校园实验服务器。
- **开发设备：**CPU i5及以上，内存8GB以上，50GB可用硬盘空间。

2.3.2 支持软件

- **客户端软件：**Chrome、Edge等现代浏览器。
- **服务器端软件：**
 - 操作系统：Linux (Ubuntu 20.04) / Windows Server / macOS
 - Web服务器：Nginx（用于前端静态资源）或后端内嵌服务器（如Spring Boot内嵌Tomcat）
 - 数据库：MySQL 5.7+ 或 PostgreSQL 12+
 - 运行时：Java 8+（若后端采用Spring Boot）或 Node.js 16+（若采用Node.js）
- **开发工具：**VS Code、IntelliJ IDEA、Git、Postman/Apifox、Navicat/DataGrip、Figma/墨刀等。

2.4 设计约束

- 必须采用前后端分离架构，前端使用Vue/React，后端使用Spring Boot/Node.js。
- 接口设计遵循RESTful风格，请求与响应数据格式统一为JSON。
- 数据库命名采用英文小写加下划线，表结构须满足第三范式。
- 支付、地图、消息推送等功能必须采用模拟方式实现，禁止接入真实第三方API。
- 代码需遵循统一的编码规范（4空格缩进、驼峰命名、必要注释等）。
- 开发过程受课程周期限制，需优先实现核心功能。

2.5 功能需求

系统需要实现以下核心功能模块：

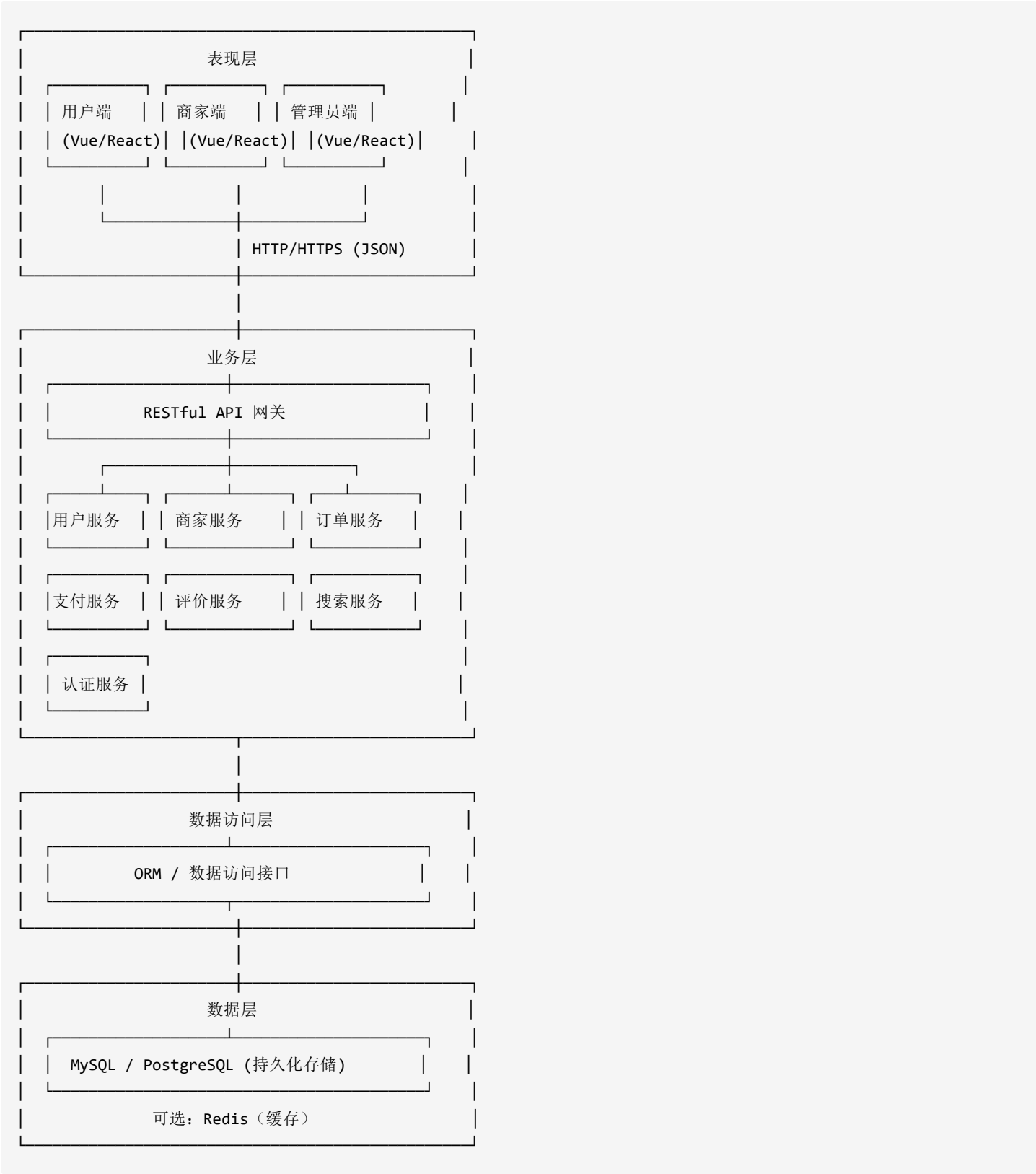
1. **用户端：**注册/登录、个人中心、商家浏览与搜索、商品详情、购物车、外卖下单、团购下单、模拟支付、订单管理（含取消、退款申请）、评价管理。
2. **商家端：**商品管理（增删改查、上下架）、订单处理（接单、出餐/备货状态）、评价回复。
3. **配送员端：**待配送订单查询、配送状态更新（取货中、配送中、已送达）。
4. **管理员端：**商家入驻审核、用户管理（查看、禁用）、评价管理（查看、删除）、数据统计（订单量、交易额、新增用户等图表展示）。
5. **公共模块：**身份认证（JWT Token）、权限控制、日志记录、异常处理。

2.6 非功能需求

- **可靠性：**订单状态流转需保证数据库事务一致性，关键操作失败时有回滚机制。
- **安全性：**用户密码加盐哈希存储；接口基于JWT进行权限校验，防止越权访问；测试数据脱敏。
- **可用性：**关键业务操作步骤简约，提供表单验证和错误提示，普通用户无需培训即可使用。
- **可维护性：**代码模块化，接口文档完整，前后端分离降低耦合。
- **可扩展性：**各业务模块独立，预留优惠券、会员、个性化推荐等扩展接口。

2.7 系统总体架构

本系统采用前后端分离的四层架构，总体架构如下图所示（以文字描述UML包图）：



各层职责：

- **表现层：**负责用户界面展示和交互，通过AJAX/Fetch调用后端API，接收JSON数据渲染页面。包含用户端、商家端、配送员端、管理员端四类前端应用（可统一项目通过路由分隔）。
- **业务层：**处理核心业务逻辑，划分为多个独立服务模块。认证服务负责登录鉴权和Token管理；用户服务管理用户信息与地址；商家服务管理人员商户、商品；订单服务管理订单状态流转；支付服务提供模拟支付功能；评价服务处理用户评价与回复；搜索服务提供商家和商品检索。

- **数据访问层**：封装对数据库的CRUD操作，通过ORM框架（如MyBatis/Spring Data JPA或Sequelize/TypeORM）实现对象与关系数据的映射，隔离业务逻辑与数据库细节。
- **数据层**：采用关系型数据库存储所有业务数据，保障事务特性。可根据需要引入Redis缓存热点数据（如商家列表），但非必需。

3 系统接口设计

3.2 接口设计

3.2.1 用户接口

- **标识**：UI-01
- **类型**：基于HTTP的Web页面交互
- **命令与语法结构**：用户通过浏览器的可视化界面进行操作，无需输入命令。系统为各类角色提供对应的操作页面：
 - 普通用户：首页（商家分类/推荐）、搜索框、商家详情页、购物车页、订单页、个人中心等。
 - 商家：商品管理页（列表、添加/编辑表单）、订单处理页、评价回复页等。
 - 配送员：待配送订单列表页、配送状态更新按钮。
 - 管理员：商家审核列表、用户管理表格、评价管理界面、统计图表仪表盘。
- **软件回答信息**：
 - 成功操作：页面提示“操作成功”，列表刷新，数据更新。
 - 失败操作：在对应字段或页面顶部显示红色错误信息，如“手机号格式错误”、“库存不足”、“网络异常，请稍后再试”。
 - 关键操作（如取消订单、删除商品）会弹出确认框。
- **硬件控制面板**：无特殊硬件控制面板，全部通过鼠标/触控和键盘输入完成。

3.2.2 与其他软件、硬件接口

- **模拟支付接口（PAY-01）**：系统内部模拟支付组件，暴露REST接口供订单服务调用。
 - 请求：`POST /api/payment/pay`，参数为 `{ "orderId": "xxx", "amount": 28.5 }`。
 - 响应：

```
{ "code": 200, "msg": "支付成功", "data": { "transactionId": "SIM20260418123456", "status": "SUCCESS" } }
```
 - 该接口以80%概率返回支付成功，20%概率返回支付失败（模拟网络波动或余额不足），不涉及真实金融交互。
- **数据库接口**：业务层通过数据访问层的ORM框架与MySQL/PostgreSQL建立连接，使用JDBC或数据库驱动，连接池配置为最大20个连接。
- **前端与后端接口**：遵循RESTful风格，使用HTTP方法（GET/POST/PUT/DELETE），所有API统一前缀 `/api/v1/`，数据格式为JSON，身份认证通过请求头 `Authorization: Bearer <token>` 传递JWT。
- **无外部硬件接口**，不依赖外部第三方服务。

3.3 系统数据结构设计

3.3.1 逻辑结构设计要点

以下列出系统主要数据结构的逻辑定义（实体及其属性），详细的数据库表结构见《数据库设计说明书》。

1. 用户（User）

- 标识符：user_id (BIGINT)
- 属性：手机号(phone)、邮箱(email)、密码哈希(password_hash)、盐值(salt)、昵称(nickname)、头像(avatar)、角色(role：USER/MERCHANT/DELIVERY/ADMIN)、状态(status：正常/禁用)、创建时间(create_time)、更新时

间(update_time)

- 关联：一个用户有多条地址记录、多个订单。

2. 用户地址 (Address)

- 标识符：address_id (BIGINT)
- 属性：用户ID(user_id)、收货人姓名(consignee)、手机号(phone)、省(province)、市(city)、区(district)、详细地址(detail)、是否默认(is_default)、创建时间
- 关联：属于一个用户。

3. 商家 (Shop)

- 标识符：shop_id (BIGINT)
- 属性：商家名称(name)、分类ID(category_id)、地址(address)、联系电话(contact_phone)、简介(description)、评分(average_rating)、月销量(monthly_sales)、审核状态(audit_status：待审核/通过/驳回)、所属用户ID(owner_user_id)、创建时间
- 关联：一个商家属于一个分类，拥有多个商品。

4. 商品 (Product)

- 标识符：product_id (BIGINT)
- 属性：商家ID(shop_id)、商品名称(name)、描述(description)、图片(image_url)、价格(price)、原价(original_price，团购时用)、库存(stock)、类型(type：外卖/团购)、状态(status：上架/下架)、有效期(valid_from, valid_to，团购用)、创建时间
- 关联：属于一个商家。

5. 订单 (Order)

- 标识符：order_id (BIGINT)
- 属性：订单号(order_sn，系统生成唯一)、用户ID(user_id)、商家ID(shop_id)、订单类型(type：外卖/团购)、总金额(total_amount)、实付金额(payment_amount)、支付方式(payment_method：余额/支付宝模拟等)、状态(status：待支付/已支付待接单/进行中/配送中/已送达/待评价/已完成/已取消/退款中/已退款)、配送地址ID(address_id)、配送员ID(delivery_user_id，可空)、支付交易流水号(transaction_id)、备注(remark)、创建时间、支付时间、完成时间
- 关联：包含多个订单行项目，关联一个用户、一个商家、一个配送员（可选）。

6. 订单行项目 (OrderItem)

- 标识符：item_id (BIGINT)
- 属性：订单ID(order_id)、商品ID(product_id)、商品名称快照(product_name)、商品图片快照(image_url)、单价(price)、数量(quantity)、小计(subtotal)
- 关联：属于一个订单。

7. 团购券 (Voucher)

- 标识符：voucher_id (BIGINT)
- 属性：用户ID(user_id)、订单ID(order_id)、商品ID(product_id)、核销码(code，唯一)、状态(status：待使用/已使用/已过期/已退款)、有效期(valid_from, valid_to)、创建时间、使用时间
- 关联：关联一个订单和一个商品。

8. 评价 (Review)

- 标识符：review_id (BIGINT)
- 属性：订单ID(order_id)、用户ID(user_id)、商家ID(shop_id)、星级(rating，1-5)、内容(content)、是否匿名(is_anonymous)、商家回复(reply)、回复时间(reply_time)、创建时间
- 关联：属于一个订单和商家。

9. 商家分类 (Category)

- 标识符：category_id (INT)
- 属性：分类名称(name)、图标(icon)、排序号(sort_order)

10. 系统日志 (OperationLog)

- 标识符：log_id (BIGINT)
- 属性：操作人ID(user_id)、操作类型(action)、描述(description)、IP地址(ip)、创建时间

3.3.2 物理结构设计要点

- **存储要求：**所有数据持久化存储在MySQL/PostgreSQL数据库中，使用InnoDB引擎（若MySQL）以支持事务。
- **访问方法：**通过主键索引进行单条记录存取；为常用查询字段（如订单号的order_sn、用户的phone、商家的category_id等）建立普通索引以加速查询。
- **存取单位：**以数据行为基本存取单位，ORM框架内部使用对象映射。
- **物理关系：**数据库部署在一台服务器上，数据文件和索引文件存放于同一存储区域。无需分区或分表，因课程项目数据量较小。
- **保密条件：**数据库访问需认证，密码文件独立配置；服务器仅接受来自应用层的连接请求。

3.3.3 数据结构与程序关系

上述数据结构分别对应业务层的各服务模块：

- **用户服务：**操作User、Address表
- **商家服务：**操作Shop、Product、Category表
- **订单服务：**操作Order、OrderItem、Voucher表
- **评价服务：**操作Review表
- **认证服务：**读取User表进行身份验证
- **管理功能：**涉及所有表的聚合查询

4 系统数据库设计

数据库采用关系型模型，遵循第三范式。核心实体关系如下（E-R简图文字描述）：

- 一个用户可有多个地址、多个订单、多个评价。
- 一个商家属于一个分类，拥有多个商品、多个订单、多个评价。
- 一个订单包含多个订单行项目，可对应一个团购券（团购订单），关联一个配送员（外卖订单）。
- 一个评价属于一个订单和一个商家。

详细的表结构、字段类型、索引及SQL脚本见《数据库设计说明书》。

5 运行设计

5.1 运行模块组合

系统运行时，各模块的组合方式依据业务流程动态调用：

1. **用户浏览与搜索流程：**表现层（用户端） --> 搜索服务 --> 数据访问层 --> 数据层（Shop/Product表）
2. **下单支付流程：**表现层 --> 订单服务（校验库存、生成订单） --> 支付服务（模拟） --> 订单服务（更新状态） --> 数据层（Order/OrderItem/Product表事务更新）
3. **配送流程：**配送员端表现层 --> 订单服务（查询待配送订单、更新状态） --> 数据层
4. **评价流程：**表现层 --> 评价服务 --> 订单服务（校验订单状态） --> 数据层（Review表写入，Shop评分更新）
5. **商家管理流程：**商家端表现层 --> 商家服务（商品管理、订单处理） --> 数据层
6. **管理员流程：**管理员端表现层 --> 聚合查询各服务或直接调用数据访问层进行统计

5.2 运行控制

- **启动顺序：**数据库服务 -> 后端Spring Boot/Node.js服务 -> 前端Nginx或开发服务器。

- **请求处理流程**：前端发送HTTP请求至后端API网关（可整合在业务层统一入口），由拦截器验证JWT Token并解析角色权限，然后路由到对应Controller，调用Service层业务逻辑，通过Repository/DAO访问数据库，返回JSON结果。
- **会话管理**：无状态JWT Token，客户端存储Token，每次请求携带，服务端验证后获取用户身份，不依赖服务器端Session。
- **并发控制**：基于数据库事务隔离级别（读已提交）和乐观锁（如给订单表加version字段）处理订单状态更新的并发冲突。
- **异常控制**：全局异常处理器捕获各种异常，返回统一错误响应。对可恢复异常（如支付失败）保留订单状态，对不可恢复异常记录日志并提示用户。

5.3 运行时间

- **用户交互操作**：简单查询响应时间<1秒，复杂关联查询（如订单列表带评价）<2秒。
- **后台批量操作**：管理员统计查询可在合理时间内完成（数据量小，预计<3秒）。
- **系统持续运行**：演示期间应能稳定运行不低于2小时，无内存泄漏等问题。
- **定时任务**（如结算无，本项目无复杂定时任务需求；若今后扩展，可安排低峰期执行）。